NNMA: An effective memetic algorithm for solving multiobjective permutation flow shop scheduling problems

Tsung-Che Chiang[a], Hsueh-Chien Cheng[b], Li-Chen Fu[b,c*]

[a]*Department of Computer Science and Information Engineering, National Taiwan Normal University, Taiwan, R.O.C.*
[b]*Department of Computer Science and Information Engineering, National Taiwan University, Taiwan, R.O.C.*
[c]*Department of Electrical Engineering, National Taiwan University, Taiwan, R.O.C.*

ABSTRACT

The permutation flow shop scheduling problem is addressed in this paper. Two objectives, minimization of makespan and total flow time, are considered. We propose a memetic algorithm, called NNMA, by integrating a general multiobjective evolutionary algorithm (NSGA-II) with a problem-specific heuristic (NEH). We take NEH as a local improving procedure in NNMA and propose several adaptations including the acceptance criterion and job-insertion ordering to deal with multiple objectives and to improve its performance. We test the performance of NNMA using 90 public problem instances with different problem scales, and compare its performance with 23 algorithms. The experimental results show that our NNMA provides close performance for 30 small-scale instances and better performance for 50 medium- and large-scale instances. Furthermore, more than 70% of the net set of non-dominated solutions is updated by NNMA for these 50 instances.

ARTICLE INFO
Keywords:
Permutation flow shop scheduling
Multiobjective optimization
Memetic algorithm
Makespan
Total flow time
NEH heuristic

* Corresponding author.

1. Introduction

Production scheduling problems have been extensively investigated in the last several decades. Many production scheduling problems have been proven to be NP-hard, which means that currently there is no polynomial-time algorithm to solve these problems optimally. This difficulty challenges researchers in the academia and attracts them to propose many optimization/approximation algorithms. On the other hand, scheduling is a critical task in the industry, too. It has great impact on the key performance indices including throughput, cycle time, and so on. Thus, managers and engineers in the shop floor also devote themselves to the scheduling problem.

In this paper, we address the scheduling problem in the permutation flow shop. The feature of a flow shop is that jobs are to be processed by machines following an identical route. In other words, each job is processed on machine 1 at first, then on machine 2, …, and finally on machine m, where m denotes the number of machines in the shop. The permutation flow shop has an additional requirement – the processing orders of jobs on all machines are the same. More specifically, all machines except the first one process jobs in a first-in-first-out (FIFO) manner. Processing times of jobs on machines are constant and known in advance. Two constraints must be satisfied to generate feasible schedules. First, each machine can process only one job at a time. Second, processing of a job j on machine k ($k \geq 2$) should start only after processing of j on machine k–1 is completed.

Two common objectives in permutation flow shop scheduling are minimization of makespan ($C_{\max}$) and total flow time (TFT). Let C_i denote the completion time of job i on machine m, makespan is defined by $C_{\max} = \max_{i=1\ldots n} C_i$, and total flow time is defined by $TFT = \sum_{i=1\ldots n} C_i$. Both objectives are well-justified in practice, since short makespan implies high throughput and resource utilization whereas short flow time

implies quick response and low work-in-process (WIP). Single-objective permutation flow shop scheduling with respect to either $C_{\max}$ or *TFT* was proven to be NP-hard and has been studied extensively (Framinan et al., 2005; Ruiz & Maroto, 2005). In the real world, however, many industries such as aircraft, electronics, semiconductor manufacturing, etc., have trade-offs in their scheduling problems where multiple objectives need to be considered to optimize the overall performance (Behnamian et al., 2009). In recent years there is a rapid growth of multiobjective metaheuristics, especially multiobjective evolutionary algorithms (Deb 2001; Coello et al., 2007). Following this research trend, many researchers in the field of production scheduling started to do studies on multiobjective scheduling (Minella et al., 2008; Lei 2009). In most multiobjective scheduling problems the concerned objectives are conflicting, which means that we usually improve one objective value at the cost of deteriorating another one. In other words, it is almost impossible to find a single optimal solution with the optimal values for all objective functions. Thus, our goal is to find a set of so-called Pareto optimal solutions. (For the definition of Pareto optimality, readers may refer to Deb 2001.) We propose a memetic algorithm (MA, also known as genetic local search, GLS) based on two well-known algorithms, NSGA-II (Deb et al., 2002) and NEH (Nawaz et al., 1983). To minimize makespan and total flow time simultaneously in the permutation flow shop, our target problem is usually denoted by $F \mid prmu \mid C_{\max}, \Sigma C_i$ in the literature.

The rest of this paper is organized as follows. Section 2 reviews existing studies on multiobjective permutation flow shop scheduling. Section 3 elaborates on the proposed NNMA. Benchmark problem instances and benchmark algorithms are described in Section 4, where we also present the experimental results of component analysis and performance comparison. Finally, we draw the conclusions and list the future research directions in Section 5.

2. Literature review

In the literature, researchers started to solve multi-objective scheduling problems by adapting single-objective algorithms to deal with multiple objectives. The most common way is to convert multiple objective values into a single value through an aggregation function. For example, Rajendran (1995) normalized makespan, total flow time, and machine idletime into the same range and summed them together as a single objective. Then a local search algorithm was proposed to minimize the aggregated objective value. Sridhar and Rajendran (1996) developed a genetic algorithm (GA) to solve the problem identical to that addressed in Rajendran (1995). Cavalieri and Gaiardelli (1998) applied a GA to solve the allocation and sequencing problem in the flow shop. Their objective was to minimize the value of a non-linear function of makespan and total tardiness. Besides aggregation functions, another way to adapt the single-objective algorithms to deal with multiple objectives is to define an optimization order. One example is the work by Neppali et al. (1996), in which a bi-objective flow shop scheduling problem was addressed by minimizing the total flow time subject to obtaining the optimal makespan.

Ishibuchi and Murata (1998) are the pioneers in solving multiobjective scheduling problems based on the concept of Pareto dominance. Different from early research studies, which usually focused on finding a single optimal solution with respect to a predefined aggregation function or optimization order, their work aimed at searching for the set of Pareto optimal solutions. They proposed a GLS algorithm and took the linear weighted sum of multiple objective values as the fitness function. The distinct feature of their algorithm is that it uses random weight vectors to do mating selection and local search. The random assignment of weight vectors leads the algorithm to search toward different directions on the objective space and thus to obtain a set of diverse solutions.

Starting from 2001, there was a rapid growth of Pareto dominance-based multiobjective evolutionary algorithms (MOEAs). Well-known MOEAs including NSGA-II (Deb et al., 2002), PESA-II (Corne et al., 2001), and SPEA2 (Zitzler et al., 2001) were developed. The appearance of these MOEAs brought about the fast advancement of the research on multiobjective scheduling. Talbi et al. (2001) conducted extensive experiments on components of MOEAs including selection and diversity maintenance mechanisms. They indicated the importance of elitism and combined fitness sharing on the genotypic and phenotypic space. Arroyo and Armentano (2005) proposed a GLS to minimize makespan, total tardiness, and maximum tardiness in permutation flow shops. The GA part of their algorithm is similar to NSGA-II. Their algorithm is featured by doing local search based on Pareto dominance in a population-based (rather than individual-based) manner. Varadharajan and Rajendran (2005) developed a multi-objective simulated annealing (MOSA) algorithm to minimize makespan and total flow time in permutation flow shops. They firstly generated two solutions (one for each objective) by sophisticated job-insertion-based heuristics. Then, the MOSA started the search process from each of these two solutions. During the search process the MOSA selects one of the two concerned objectives as the acceptance criterion following a probability function. This function is varied in a way so that the entire objective space is covered uniformly. Pasupathy et al. (2006) followed Varadharajan and Rajendran (2005) and proposed the PGA-ALS algorithm. PGA-ALS also applies sophisticated heuristics to generate initial solutions, and the GA part in PGA-ALS is also similar to NSGA-II. An archive of non-dominated solutions is maintained, and each individual in the archive is subjected to the local search with insertion and swap as the neighborhood functions. Geiger (2007) had a study on the problem structure of multiobjective permutation flow shop scheduling problems. He found that the Pareto optimal solutions

concentrate in the search space and form a “big-valley” structure just like in some single-objective optimization problems.

Besides Pareto dominance-based algorithms, there are also more and more aggregation-based algorithms for multiobjective scheduling problems. Ishibuchi et al. (2003) proposed an improved version of the MOGLS by Ishibuchi and Murata (1998). In the improved version, they chose only good individuals to do local search and determined suitable search directions with the aid of tournament selection. Chang et al. (2002) proposed the gradual-priority weighting (GPW) GA for permutation flow shop scheduling. The linear weighted summation of concerned objectives was taken as the fitness function. The difference from the MOGLS by Ishibuchi & Murata (1998) is in that the weight vector used at each generation is unique in GPWGA. By adjusting the weight vector periodically, GPWGA searches for the optimal solutions following the direction of one objective at the beginning and towards other objectives until the end. Later, Chang et al. (2007) focused on minimizing makespan and total tardiness in the permutation flow shop by using a sub-population GA. They kept the linear weighted sum method, divided the population into sub-populations, and assigned different weight vectors to the sub-populations. Li and Wang (2007) proposed a hybrid GA for multiobjective flow shop scheduling. Their algorithm is composed of a quantum-inspired GA (QGA) and a permutation-based GA (PGA). The QGA is responsible for exploration and assigns fitness values by linear-weighted-sum function, whereas the PGA is responsible for exploitation and assigns fitness values by non-dominated sorting. The experimental results showed that the hybrid of QGA and PGA provides better performance than pure PGA.

Recently Minella et al. (2008) conducted a comprehensive review and evaluation of 23 algorithms. These algorithms include aforementioned well-known general algorithms, e.g. NSGA-II, SPEA2, PESA-II, and IBEA (Zitzler & Künzli, 2004), and

specific algorithms, e.g. ENGA (Bagchi, 2001), MOGLS (Arroyo & Armentano, 2005), MOSA (Varadharajan & Rajendran, 2005), PGA-ALS (Pasupathy et al., 2006), and PILS (Geiger, 2007). They implemented these algorithms to solve the permutation flow shop scheduling problem regarding three two-criterion combinations. In their study, the MOSA (Varadharajan & Rajendran, 2005) and the MOGLS (Arroyo & Armentano, 2005) were the best performers. We will compare our proposed algorithm with these 23 algorithms in Section 4.

3. Proposed NNMA

3.1 Overview

In this section we will present the details of the proposed NNMA. We first give an overview of the algorithm in this subsection and will explain each component in the following subsections. The NNMA follows the framework of a typical MA. In each generation we do mating selection, crossover, mutation (probabilistically), evaluation, and environmental selection to produce the population for the next generation. Evaluation of individuals is achieved by decoding the chromosomes into corresponding schedules, then calculating the concerned objective values, and at last assigning the fitness values. Local improving is done periodically (every T_L generations). The entire algorithm stops when the maximum number of generations (T_G) or the predefined time limit is reached.

<< Insert Algorithm 1 about here >>

3.2 Chromosome encoding and decoding

We encode each solution by a permutation of jobs, which is a natural and popular encoding mechanism for solving the permutation flow shop scheduling problem. Given a permutation of jobs, we decode it by scheduling the jobs one by one from left

to right onto the machines. Assuming that there are four jobs and four machines in the flow shop, an example of decoding an individual with a chromosome [1 2 3 4] is depicted in Fig. 1.

<< Insert Fig. 1 about here >>

3.3 Fitness assignment and mating selection

After chromosomes are decoded and the concerned objectives ($C_{\max}$ and TFT) are obtained, we assign fitness values to them to reflect their importance in the evolutionary process. Various mechanisms for fitness assignment considering multiple objectives have been proposed, and we choose one of the most recognized mechanisms, the mechanism of NSGA-II (Deb et al., 2002). It assigns the fitness value based on the level of non-domination (also known as Pareto ranking) and the crowding distance. Due to the limitation of space, we do not present its details here. Readers who are interested in NSGA-II may refer to Deb et al. (2002).

When selecting individuals as parents to produce the offspring, we favor those with higher solution quality and diversity by the 2-tournament selection. It selects two random individuals and takes the one with lower level of non-domination (higher solution quality) and larger crowding distance (higher diversity) as the parent.

3.4 Crossover and mutation

Crossover and mutation are two primary means by which the parents produce the offspring in the evolutionary algorithm. There are many kinds of crossover and mutation operators for the permutation-based encoding. We use the two-point linear order crossover (LOX) and the swap mutation. The two-point LOX first chooses two cut points randomly and passes the section enclosed by the cut points from one parent to one offspring. Next, the un-passed job indices are put in the unfilled positions in

the order of their occurrence in the other parent. Given two parents 1<u>234</u>56 and 5<u>41</u>632, for example, assuming that the cut points are at positions two and four, the two offspring produced by LOX are 5<u>234</u>16 and 2<u>41</u>635.

The swap mutation chooses two genes randomly and swaps their values. Assuming that the length of the chromosome is n, we do n trials of the swap mutation to each offspring produced by LOX. The success rate of each trial is r_m. In other words, $n \cdot r_m$ pairs of genes are swapped on average.

3.5 Environmental selection

Most environmental selection mechanisms follow the principle of "survival of the fittest" to raise the solution quality and adopt strategies like immigration (replacing current individuals with randomly-created individuals) to keep the population diversity. Here we use the 2/4-replacement mechanism. After two offspring are produced by LOX and swap mutation, we assign fitness values to them and their parents. Then, among these four individuals the two ones with the highest fitness values and distinct objective values will replace the two parents in the population. This mechanism implicitly implements the elitism; in addition, doing the replacement avoids the rapid growth of individuals with similar gene structures and the loss of population diversity. It was shown to provide good performance in our previous work (Chiang & Fu, 2008). To avoid the loss of non-dominated solutions, we also have an archive to store all non-dominated solutions generated during the execution of MA. Every time when two offspring are produced by LOX and swap mutation, we will check if they should be stored in the archive and if any solutions in the archive should be removed. In fact, this archive represents the output of our NNMA.

3.6 NEH-based local improving procedure

GA and local search are integrated in MA to obtain the advantages of both algorithms, the exploration ability of GA and the exploitation ability of local search. The neighborhood function, which generates neighboring solutions around the current solution, is a key component in the local search procedure. Because local search aims at searching intensively in a (small) promising region on the solution space, it usually generates neighboring solutions by making a small modification to the current solution. In the literature, two popular neighborhood functions are the insertion function (Ishibuchi & Murata, 1998; Talbi et al., 2001; Arroyo & Armentano, 2005; Varadharajan & Rajendran, 2005; Cheng et al., 2008) and the swap function (Ishibuchi et al., 2003; Pasupathy et al. 2006; Geiger 2007; Cheng et al., 2008). In our preliminary experiments, however, we found that better solutions can be generated by using a different neighborhood function.

In this work we propose an NEH-based neighborhood function. The NEH heuristic (Nawaz et al., 1983) is prominent for minimizing makespan in the permutation flow shop. It is a constructive, iterative, and greedy procedure based on the insertion neighborhood. Given n jobs, it firstly marks all jobs as unscheduled and sorts all jobs in non-increasing order of total processing time. Then, it follows this order and inserts unscheduled jobs one by one into the best partial sequence of scheduled jobs. At iteration t, the NEH inserts the tth unscheduled job into t possible positions in the best partial sequence obtained at iteration (t–1) while keeping the relative orders of the previous (t–1) scheduled jobs unchanged. Among the t newly generated partial sequences, the one whose corresponding schedule has the shortest makespan is the best partial sequence at iteration t. Then, the inserted job is marked as scheduled. The NEH repeats the above steps for n times (t = 1, 2, …, n) to construct a complete schedule, which usually has short makespan.

3.6.1 Overview

A common integration of the NEH heuristic and metaheuristics is using NEH to generate initial solutions for metaheuristics (Sridhar & Rajendran, 1996; Rajendran & Ziegler, 2004; Varadharajan & Rajendran, 2005; Pasupathy et al., 2006). These studies used NEH to generate a single solution that is particular good for a certain objective, e.g. makespan, at a time. Some researchers (Framinan et al., 2002; Arroyo and Armentano, 2004) applied NEH to generate a set of non-dominated solutions. However, their approaches only took NEH as a stand-alone procedure and did not consider combining NEH with other metaheuristics. Noticing the exploitation ability of NEH and exploration ability of GA, we integrate NEH and GA more tightly – we propose a MA with a NEH-based local improving procedure.

Before going into the details of the proposed local improving procedure, we use Fig. 2 as an example to show its basic idea. To do local improving to an individual, first we randomly choose two cut points and extract the segment of jobs enclosed by the cut points. Here, jobs 3 and 4 are extracted. The relative orders of non-extracted jobs 1, 2, and 5 will be unaltered hereafter. Extracted jobs 3 and 4 will be re-inserted following a certain order. Here, we assume that job 3 is re-inserted first. At the first iteration, job 3 is re-inserted into four possible positions in the partial sequence [1 2 5]. We assume that ($C_{\max} + TFT$) is chosen as the acceptance criterion to deal with both objectives simultaneously. Accordingly, the second partial sequence [1 3 2 5] is the best partial sequence. At the next iteration, job 4 is re-inserted into five possible positions in [1 3 2 5]. Since our goal is to find the Pareto optimal solutions, we will collect all non-dominated solutions among the solutions produced in the last iteration. In the example, solutions [4 1 3 2 5], [1 3 4 2 5], and [1 3 2 5 4] are collected.

<< Insert Fig. 2 about here >>

The algorithm of the local improving procedure is given in the following. We will discuss three main functions *Segment*(), *Reorder*(), and *Accept*() in the next three subsections. We will address the connections between the GA part and the local improving part by discussing selection of individuals to do local improving, refilling of population, and timing to do local improving in the last subsection.

<< Insert Algorithm 2 about here >>

3.6.2 Segmentation

The standard NEH starts with an empty set of scheduled jobs; in other words, all jobs are extracted and re-inserted to construct a solution. In our algorithm, one major difference from the conventional combination of NEH and GA is that we do not use NEH as a stand-alone procedure. Instead, we apply NEH inside GA to improve the population, which is simultaneously evolving during the execution of GA. With the evolving ability of GA, we expect that most parts of the chromosomes (job permutations) are already satisfactory with respect to the concerned objectives. Besides, re-constructing the entire solution by re-inserting all jobs is computationally expensive and will deteriorate the efficiency of our algorithm. Thus, we utilize NEH to adjust only part (not all) of the chromosome. We extract a set of jobs and re-insert them back while maintaining the relative positions between the non-extracted jobs. The number of extracted jobs, n_{NEH}, is controlled by the parameter N_{NEH} (the maximum number of extracted jobs) and the evolution progress. Based on our experience, it is a good strategy to do exploration (using a large neighborhood) at the beginning of the evolution and to do exploitation (using a small neighborhood) near the end of the evolution. Thus, we define the number of extracted jobs by $n_{\mathrm{NEH}} = \max\{N_{\mathrm{NEH}}\cdot(1 - t/T_{\mathrm{G}}),\ 1\}$, where t is the current generation number and T_{G} is the maximum number of generations of MA. To decide the set of extracted jobs, we first

select a random position, say i, in the chromosome. Then, jobs from position i to $((i + n_{\mathrm{NEH}} - 1) \bmod n)$ are extracted, where n is the length of the chromosome. An example is given in Fig. 3.

<< Insert Fig. 3 about here >>

3.6.3 Acceptance strategy

Another difference in our algorithm from the typical use of NEH is that we are dealing with multiple objectives. In the standard NEH, the best partial sequence is the one with the shortest makespan. In our algorithm, we propose three strategies to decide the best partial sequence(s).

Strategy 1 (Random Objective, RO): To search for the Pareto optimal solutions regarding multiple objectives, one simple strategy is to search according to a single objective function during a single search and change the objective function among different searches. In this strategy we choose $C_{\max}$ or TFT before applying NEH to each individual. Suppose $C_{\max}$ (TFT) is chosen, then, at all but the last iteration in NEH, the partial sequence with the shortest $C_{\max}$ (TFT) is taken as the best one. If there is more than one partial sequence with the shortest $C_{\max}$ (TFT), the first visited one is taken as the best one.

Strategy 2 (linear weighted sum with Distributed Weights, DW): Comparing with the RO strategy, a more advanced strategy is to search along different directions according to the locations of individuals on the objective space. In this strategy, we first sort the individuals in increasing order of $C_{\max}$. The weight vector w_i associated with the ith sorted individual is defined by $w_i = (\ 1 - (i-1)/(|S|-1),\ (i-1)/(|S|-1)\ )$, where S is the set of individuals to do local improving. Then, at all but the last iteration in NEH, the partial sequence with the smallest $F_i = w_{i1} \cdot C_{\max}' + w_{i2} \cdot TFT'$ is taken as the best one, where $C_{\max}'$ and TFT' denote normalized values of $C_{\max}$ and TFT,

respectively. Here we do not use the original $C_{\max}$ and *TFT* since they may have quite different scales. We normalize the values of $C_{\max}$ and *TFT* of each partial sequence through dividing them by the values of $C_{\max}$ and *TFT* of the first generated partial sequence at the same iteration, respectively. If there is more than one partial sequence with the smallest F_i, the first visited one is taken as the best one.

Strategy 3 (Pareto Dominance, PD): In the previous two strategies, exactly one best partial sequence is chosen at each iteration in NEH. (Referring to Algorithm 2, the size of the set *B* is always one.) To broaden the search scope, the third strategy is to take all non-dominated partial sequences as the best partial sequences.

We use Table 1 to summarize how the three proposed acceptance strategies work. Fig. 4 is provided to catch the basic idea of the three acceptance strategies. The effects of these strategies on the algorithm performance will be discussed in Section 4.4.1.

<< Insert Table 1 and Fig. 4 about here >>

3.6.4 Ordering strategy

The third difference between our local improving procedure and the standard NEH is that we try to insert the extracted jobs following various ordering strategies. The standard NEH inserts the jobs following the so-called LPT (longest total processing time first) order. In this study we consider the LPT ordering strategy as well as other six strategies.

Evolution strategy: The order of extracted jobs in the permutation originally recorded on the individual is used. In other words, there is no re-ordering.

LPT strategy: The extracted jobs are sorted in non-increasing order of total processing times.

SPT strategy: The extracted jobs are sorted in non-decreasing order of total

processing times.

Random strategy: The extracted jobs are shuffled into a random order.

Uniform strategy: One of the above four strategies is selected randomly and then applied.

Cyclic strategy: The first four strategies are applied in a cyclic manner. In other words, each strategy is applied every four times of calling the local improving procedure.

Stuck strategy: When the local improving procedure finds no new non-dominated solutions with a certain strategy, it uses the next strategy at the next time. Only the first four strategies are considered in the rotation cycle.

The effects of these strategies on the algorithm performance will be discussed in Section 4.4.3.

3.6.5 Connections between GA and local improving

Every time we do local improving to an individual, we need to evaluate $(n - n_{\mathrm{NEH}} + 1) + (n - n_{\mathrm{NEH}} + 2) + \ldots + n = 0.5 \cdot n_{\mathrm{NEH}} \cdot (2 \cdot n - n_{\mathrm{NEH}} + 1)$ partial sequences, where n denotes the chromosome length. (This is the number of evaluations required by the RO or DW acceptance strategy. We need more evaluations if we use the PD strategy.) It can be a large computational effort. Therefore, we do not apply local improving to every individual at every generation. In our algorithm, we apply local improving to a selected set of individuals every T_{L} generations. We propose three strategies to select the individuals to do local improving. These strategies have different degree of preference for individuals with higher fitness. We will discuss their effects on the algorithm performance in Section 4.4.2.

Archive strategy: Only the non-dominated solutions stored in the archive A (see Section 3.5) can do local improving.

2-tournament strategy: We use 2-tournament selection to pick up $|A|$ individuals from the current population to do local improving.

Random strategy: We pick up $|A|$ individuals randomly from the current population to do local improving.

At the end of the local improving procedure, we have to put the found non-dominated solutions back into the population to continue the evolutionary process. This is the task of function *Refill*() in Algorithm 1. In our algorithm, we replace the best $|S^*|$ individuals in the original population with the set of non-dominated solutions S^* output by the local improving procedure.

3.7 Generation of initial population and stopping criteria

The individuals in the initial population are generated randomly. Every individual is a random permutation of n indices of jobs. The entire algorithm stops when the maximum number of generations (T_G) or the predefined time limit is reached.

4. Experiments and results

4.1 Benchmark algorithms and benchmark problem instances

Problem instances in our experiments are taken from Taillard's data set (Taillard 1993), which is one of the most widely used benchmark data sets for shop scheduling. We take 90 problem instances from the instances designed for flow shop scheduling. These instances were usually denoted by TA01–90 in the literature. Every ten instances having the same number of jobs and of machines form a problem category. The problem scale ($n \times m$) of these nine categories are 20×5, 20×10, 20×20, 50×5,

50×10, 50×20, 100×5, 100×10, and 100×20. These instances were taken as testing instances in many existing studies (Varadharajan & Rajendran, 2005; Pasupathy et al., 2006; Guo et al., 2007; Minella et al., 2008), and the net sets of non-dominated solutions for these instances are available either in the published papers or on the author's website. To verify the performance of the proposed NNMA, we will compare the net sets of non-dominated solutions generated by NNMA with those generated by Minella et al. (2008). In Minella et al.' study, the net sets of non-dominated solutions were generated by 23 existing algorithms. We got these solutions from the personal website of the second author of that paper.

4.2 Performance metrics

We adopt three metrics to do performance evaluation. The first two metrics, I_ε and I_{H}, are used following the instructions in Minella et al. (2008). Given two sets, A and B, of solutions, the binary epsilon indicator $I_\varepsilon(A, B)$ equals the minimum factor ε such that any solution in B is ε-dominated by at least one solution in A. (For more details about ε-dominance, see Laumanns et al., 2002.) In the experiment, we calculate the $I_\varepsilon(A, R)$ value for the net set A of non-dominated solutions obtained by each tested algorithm, where R is the net set of non-dominated solutions obtained by all tested algorithms. The objective values of each solution x are normalized into values in the interval [1, 2] by $f_j^{'}(x) = (f_j(x) - f_j^{\min})/(f_j^{\max} - f_j^{\min}) + 1$, where $f_j(x)$ denotes the jth objective value of x and $f_j^{\min}/f_j^{\max}$ denotes the minimum/maximum of the jth objective value among the solutions in the union of the net sets of non-dominated solutions obtained by all tested algorithms. The normalized I_ε indicator is defined by $I_\varepsilon(A, R) = \max_{y\in R} \min_{x\in A} \max_{1\le j\le 2} \{f_j^{'}(x)/f_j^{'}(y)\}$. It takes values between 1 and 2, and a small I_ε value indicates better solution quality.

The second metric, the hypervolume indicator I_H, measures the area covered by a set of non-dominated solutions in the case of two objectives. The I_H values are calculated based on the objective values normalized into [0, 1]. The reference point is [1.2, 1.2], and hence the maximum I_H value is $1.2^2 = 1.44$.

The third metric I_{PF} is the fraction of Pareto front that is covered by the net set A of non-dominated solutions obtained by each tested algorithm. Since the true Pareto front is not known, we use the reference set R instead. It is defined by $I_{PF}(A) = |A \cap R|/|R|$. A larger value for the I_H and I_{PF} indicators means better solution quality. Fig. 5 illustrates calculation of the three performance metrics.

<< Insert Fig. 5 about here >>

4.3 Computing environment & experimental settings

The proposed NNMA was implemented in C++ with Microsoft Visual Studio 2005. The test was conduced on personal computers running Windows XP with Intel 3.0 GHz CPU and 2 GB RAM. Each version of algorithm solved each problem instance for ten times. The net set of non-dominated solutions obtained over ten runs was then taken to evaluate each version of algorithm with respect to the three aforementioned performance metrics. The time limit for problem instances with n jobs and m machines is $(0.5 \cdot n \cdot m \cdot 0.2)/(3.0/2.4)$ seconds. The term $(0.5 \cdot n \cdot m \cdot 0.2)$ is the time limit in Minella et al. (2008). We divide it by (3.0/2.4) because Minella et al. used PC with 2.4 GHz CPU and we used PC with 3.0 GHz CPU.

4.4 Performance analysis of the proposed NNMA

We investigate the effects of different strategies for the acceptance criterion, selection of individuals, and ordering of extracted jobs in the local improving procedure in the first three subsections. The setting of other parameters of NNMA is

discussed in the first subsection. We used 30 problem instances, TA11–20 (20×10), TA41–50 (50×10), and TA71–80 (100×10) in the experiments in the first three subsections. In the fourth subsection we use all 90 problem instances to do performance comparison between NNMA and the 23 benchmark algorithms. In the last subsection we give observations on the curves of Pareto fronts of the 90 problem instances.

4.4.1 Effects of parameters and the acceptance criterion in the local improving procedure

In the first experiment we want to set suitable values for the parameters of NNMA. Since we put the focus on the local improving procedure in this study, we set values for the parameters of the GA part according to our experience. As for the parameters related to the local improving procedure, we tested three values (10, 20, and 40) for T_L (The local improving procedure is invoked every T_L generations.) and three values ($0.05 \cdot n$, $0.1 \cdot n$, and $0.2 \cdot n$) for N_{NEH} (the maximum number of extracted jobs). As we mentioned in Section 3.6.2, the number of extracted jobs in the local improving procedure is initialized by N_{NEH} and decreases gradually during the execution. We call it the D-strategy for its dynamic adjustment of the number of extracted jobs. To see the effect of the D-strategy, we also tested another strategy, which fixes the number of extracted jobs by N_{NEH} throughout the execution of the MA. We call it the S-strategy for its static nature. The three acceptance strategies mentioned in Section 3.6.3 were tested in this experiment as well. We used the Archive strategy to select the individuals to do local improving and the Evolution strategy to determine the order of extracted jobs to re-insert in local improving. We tested 54 (3×3×2×3) versions of NNMA with different combinations of parameter values and strategies. The tested values for the parameters and the strategies are summarized in Table 2.

<< Insert Table 2 about here >>

For each problem category, we select the best algorithm setting among the 54 versions according to their I_ε and I_H values. Due to the limitation of space, we do not present the detailed I_ε and I_H values of each tested version. Instead, we mark the value of each parameter in the best setting in boldface in Table 2. For the values of T_L and N_{NEH} and the use of dynamic adjustment of the number of extracted jobs in NEH, the best algorithm setting for solving larger-scale problem instances favors smaller T_L (i.e. more invocations of local improving), smaller N_{NEH} (i.e. smaller neighborhood), and the D-strategy. All these reveal that stronger exploitation ability is required to solve larger-scale problem instances within a time limit. The DW strategy is always the best acceptance strategy regardless of the problem scale. Comparing with the RO strategy, DW improves the individuals following adequate search directions determined by the individuals' locations on the objective space. Comparing with the PD strategy, DW improves the individuals following one definite search direction and saves computation time to let NNMA run for more generations. We think of these as the reasons why DW outperforms the other two strategies.

4.4.2 Effects of selection of individuals to do local improving

In the second experiment, we used the best algorithm setting found in the first experiment and investigated the effects of different ways to select individuals for local improving. In Section 3.6.5, we propose three strategies – Archive, 2-tournament, and Random. Their average performance in terms of I_ε and I_H metrics are summarized in Table 3.

<< Insert Table 3 about here >>

When smaller-scale problem instances are solved, the 2-tournament strategy provides better performance. As the problem scale increases, the Archive strategy

becomes the best choice. In particular, the performance difference increases significantly when the number of jobs increases from 50 to 100. These results show that an appropriate degree of exploration is beneficial for solving small-scale problems. However, when the problem scale gets large, strong exploitation is again more useful.

4.4.3 Effects of ordering strategies in the local improving procedure

In the third experiment we used the best algorithm setting identified in the second experiment, and we want to know if there is a better order than LPT (the order which is followed by the standard NEH) to re-insert the extracted jobs in the local improving procedure. In Section 3.6.4 we propose six other ordering strategies. Their average performance in terms of I_ε and I_H metrics are summarized in Table 4.

<< Insert Table 4 about here >>

The results show that problem instances with different scale favor different strategies. All in all, the Stuck strategy is the best among seven tested strategies since it is at the 2nd, 1st, and 3rd place in the three problem categories, respectively. It suggests a good way to combine multiple strategies. Surprisingly, we found that the LPT strategy is always the worst one. It implies that a strategy that is good in a certain objective might not still be a good one when multiple objectives are considered.

4.5 Performance comparison with benchmark algorithms

In the previous three experiments we determined the best algorithm settings for problem categories TA11−20 (20×10), TA41−50 (50×10), and TA71−80 (100×10), respectively. In the fourth experiment we compared NNMA with 23 benchmark algorithms in Minella et al. (2008) (we called it Combo23 hereafter). We applied the best algorithm setting for solving n×10 problem instances to solve n×5 and n×20

instances. The only difference is the time limit. (Recall that the time limit for solving $n \times m$ instances is $(0.5 \cdot n \cdot m \cdot 0.2)/(3.0/2.4)$ seconds, see Section 4.3.) The average values of I_ε, I_H, and I_{PF} metrics for all nine problem categories are reported in Table 5.

<< Insert Table 5 about here >>

From Table 5, we see that NNMA totally outperforms Combo23 in six problem categories and is totally outperformed in only one problem category. In the remaining two problem categories one algorithm outperforms the other in terms of some but not all three performance metrics. Performances of NNMA and Combo23 are close when they solve small-scale problem instances (TA01–30, 20 jobs). However, NNMA significantly outperforms Combo23 when they solve medium-scale problem instances (TA31–60, 50 jobs). The solutions found by Combo23 only cover 2 ~ 13% of the net set of non-dominated solutions. In other words, NNMA updates a very large portion (87% ~ 98%) of the net set of non-dominated solutions. The average I_ε value of Combo23 is between 1.078 and 1.161, which means that the deviation percentage of the normalized objective values of its solutions from the normalized objectives values of the solutions in the reference set is at least 7% and at most 16%. By contrast, the average I_ε value of NNMA is between 1.002 and 1.011, which means that the deviation percentage is no more than 1.1%.

When NNMA and Combo23 solve large-scale instances (TA61–90, 100 jobs), Combo23 is better in one category (TA61–70, 5 machines) while NNMA is better in the other two. NNMA updates 71% and 99% of the net sets of non-dominated solutions for the two problem categories with ten and twenty machines, respectively. To explain the superior performance of Combo23 for the TA61–70 problem category, we conjecture that some of the 23 benchmark algorithms have sophisticated heuristics to generate high-quality initial solutions for problem instances with many jobs and few machines. We will do more investigation on this issue in our future work.

In Table 6, we also provide the average deviation percentage of the minimum $C_{\max}$ and *TFT* values of the solutions obtained by NNMA from the best known solutions reported in Zobolas et al. (2009), Zhang et al. (2009), and Tseng & Lin (2009). The average deviation percentage for $C_{\max}$ is only 0.17%, 0.85%, and 0.98% for small- (20 jobs), medium- (50 jobs), and large-scale (100 jobs) instances, respectively. For the *TFT* values, NNMA found the best solutions for 27 among 30 small-scale instances. The average deviation percentage is 0.62% and 1.31% for medium- and large-scale instances.

<< Insert Table 6 about here >>

In summary, we compare the solutions obtained by the proposed NNMA with those obtained by 23 multiobjective algorithms using 90 problem instances. Our NNMA provides close performance for 30 small-scale instances and much better performance for 50 medium- and large-scale instances. Most of the 23 benchmark algorithms focused on the fitness assignment and/or control of searching directions but not the way to generate new solutions. In this work, we try another research direction. We propose an NEH-based neighbourhood function in the local improving procedure in NNMA. The experimental results confirm that the proposed local improving procedure is useful to find high-quality solutions, especially when dealing with larger-scale problem instances. Furthermore, the average deviation percentage of minimum $C_{\max}$ and *TFT* values provided by NNMA from the best known solutions is usually below 1% and no more than 2.22%. Note that these best known solutions are generated by algorithms dedicated to a certain objective. By contrast, NNMA shows the ability to minimize two objectives simultaneously. Due to the limitation of space, we only provide the net sets of non-dominated solutions for instances TA41–50, TA51–60, and TA81–90 in Table 7, 8, and 9, respectively. (We update at least 96% of the net set for these three problem categories.) Readers who are interested in

comparing their algorithms with ours are welcome to contact the first author to get the net sets for all 90 instances.

<< Insert Table 7, 8, and 9 about here >>

4.6 The distribution of solutions in net sets of non-dominated solutions

In the literature on multiobjective shop scheduling, quite few studies provided the distribution of solutions in the net sets of non-dominated solutions for the benchmark problem instances. In this last subsection we want to share our observations. We plotted the net sets of non-dominated solutions on the 2-D objective space for all 90 problem instances. Six typical distributions are selected and shown in Fig. 6.

<< Insert Fig. 6 about here >>

The first type of distribution (e.g. the distribution of the net set of non-dominated solutions obtained for instance TA28) is the one that most fits our imagination, a smooth (roughly) convex curve. The second type of distribution (e.g. TA72) contains large gaps between the solutions in the knee part and those in the extreme parts. The gap may sometimes appear at the extreme part along only one objective. It implies the difficulty in searching from the knee part toward the extreme part. The third and fourth types of distributions show that for some problem instances optimizing one objective is relatively easier than optimizing the other. This suggests that sometimes it might be a good strategy to divide the search efforts "unevenly" with respect to the concerned objectives. The fifth and six types of distributions show that for some problem instances the non-dominated solutions can cluster together. This draws our attention to the ability of algorithm to explore and to avoid loss of population diversity. We hope these figures and observations could motivate readers for new designs and ideas of MOEAs. We will also do more investigations and improve our algorithm according to these observations in the future researches.

5. Conclusions

In this paper we address the permutation flow shop scheduling problem with minimization of makespan and total flow time as the objectives. A general multiobjective evolutionary algorithm, NSGA-II, is integrated with a problem-specific heuristic, NEH, to come up with a memetic algorithm, NNMA. The idea of NEH is taken to develop a local improving procedure in NNMA, and several adaptations are done to raise the performance. We not only propose suitable strategies (dynamic segmentation strategy, DW acceptance strategy, and Stuck ordering strategy) in the local improving procedure, but we also investigate the ways (timing of invocation of local improving and selection of individuals to do local improving) to connect the evolutionary process and the local improving procedure. The proposed NNMA is compared with 23 algorithms using 90 public problem instances. It provides close performance for 30 instances and significantly better performance for 50 instances. For these 50 instances, more than 71% of the net set of non-dominated solutions is updated. The experimental results confirm the success of the proposed integration of NSGA-II and NEH in NNMA.

Our future researches will include the following topics: First, we will do more investigations on the population initialization procedure. Sophisticated procedures like that proposed by Varadharajan and Rajendran (2005) will be studied. Second, we will think about more design ideas according to our observations on the distributions of non-dominated solutions. Third, we will combine the NEH-based local improving with recently proposed multiobjective evolutionary algorithms, such as MOEA/D (Zhang & Li, 2007), to seek for further performance improvement.

Acknowledgment

This research was supported by National Science Council of Republic of China under research grant No. NSC 98-2221-E-003-012 and No. 97-3114-E-002-002.

References

Arroyo, J.E.C., & Armentano, V.A. (2004) A partial enumeration heuristic for multi-objective flowshop scheduling problems. *Journal of the Operational Research Society, 55*, 1000-1007.

Arroyo, J.E.C., & Armentano, V.A. (2005) Genetic local search for multi-objective flowshop scheduling problems. *European Journal of Operational Research, 167*, 717-738.

Bagchi, T.P. (2001) Pareto-optimal solutions for multi-objective production scheduling problems. In E. Zitzler et al. (Eds.), *First International Conference on Evolutionary Multi-Criterion Optimization, EMO*, *LNCS, 1993*, (pp. 458-471). Springer, Heidelberg.

Behnamian, J., Fatemi Ghomi, S.M.T., & Zandieh, M. (2009) A multi-phase covering Pareto-optimal front method to multi-objective scheduling in a realistic hybrid flowshop using a hybrid metaheuristic. *Expert Systems with Applications, 36*, 11057-11069.

Cavalieri, S., & Gaiardelli, P. (1998) Hybrid genetic algorithms for a multiple-objective scheduling problem. *Journal of Intelligent Manufacturing, 9*, 361-367.

Chang, P.-C., Hsieh, J.-C., & Lin S.-G. (2002) The development of gradual-priority weighting approach for the multi-objective flowshop scheduling problem. *International Journal of Production Economics, 79*, 171-183.

Chang, P.-C., Chen, S.-H., & Liu, C.-H. (2007) Sub-population genetic algorithm with mining gene structures for multiobjective flowshop scheduling problems. *Expert Systems with Applications, 33*, 762-771.

Cheng, H.C., Chiang, T.C., & Fu, L.C. (2008) Multiobjective permutation flowshop scheduling by an adaptive genetic local search algorithm. In *IEEE Congress on Evolutionary Computation*, (pp. 1596-1602).

Chiang, T.C., & Fu, L.C. (2008) A rule-centric memetic algorithm to minimize the number of tardy jobs in the job shop. *International Journal of Production Research, 46*, 6913-6931.

Coello, C.A.C., Van Veldhuizen, D.A., & Lamont, G.B. (2007) *Evolutionary algorithms for solving multi-objective problems*. Kluwer Academic Publishers.

Corne, D.W., Jerram, N.R., Knowles, J.D., & Oates, M.J. (2001) PESA-II: region-based selection in evolutionary multiobjective optimization. In Spector et al. (Eds.) *Genetic and Evolutionary Computation Conference, GECCO-2001*, (pp. 283-290).

Deb. K. (2001) *Multiobjective optimization using evolutionary algorithms.* Wiley, U.K.

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation, 6*, 182-197.

Framinan, J.M., Leisten, R., & Ruiz-Usano, R. (2002) Efficient heuristics for flowshop sequencing with the objectives of makespan and flowtime minimization. *European Journal of Operational Research, 141*, 559-569.

Framinan, J.M., Leisten, R., & Ruiz-Usano, R. (2005). Comparison of heuristics for flowtime minimization in permutation flowshops. *Computers & Operations Research*, *32*, 1237-1254.

Geiger, M.J. (2007) On operators and search space topology in multi-objective flow shop scheduling. *European Journal of Operational Research, 181*, 195-206.

Guo, W., Chen, G., Huang, M., & Chen, S. (2007) A discrete particle swarm optimization algorithm for the multiobjective permutation flowshop sequencing problem. In Cao (Ed.) *Fuzzy Information and Engineering, 40*, (pp. 323-331).

Ishibuchi, H., & Murata, H. (1998) A multi-objective genetic local search algorithm and its application to flowshop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics – Part C, 28*, 392-403.

Ishibuchi, H., Yoshida, T., & Murata T. (2003) Balance between genetic search and local search in memetic algorithms for multiobjective permutation flowshop scheduling. *IEEE Transactions on Evolutionary Computation, 7*, 204-223.

Laumanns, M., Thiele, L., Deb, K., & Zitzler, E. (2002) Combining convergence and diversity in evolutionary multiobjective optimization. *Evolutionary Computation, 10*, 263–282.

Lei, D. (2009) Multi-objective production scheduling: a survey. *International Journal of Advanced Manufacturing Technology*, *43*, 926-938.

Li, B.-B., & Wang L. (2007) A hybrid quantum-inspired genetic algorithm for multiobjective flow shop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics – Part B, 37*, 576-591.

Minella, G., Ruiz, R., & Ciavotta, M. (2008) A review and evaluation of multi-objective algorithms for the flowshop scheduling problem. *INFORMS Journal of Computing, 20,* 451-471.

Nawaz, M., Enscore, Jr.E.E., & Ham, I. (1983) A heuristic algorithm for the m-machine, n-job flowshop sequencing problem. *OMEGA, 11*, 91-95.

Neppalli, V.R., Chen, C.-L., & Gupta, J.N.D. (1996) Genetic algorithms for the two-stage bicriteria flowshop problem. *European Journal of Operational Research, 95,* 356-373.

Pasupathy, T., Rajendran, C., & Suresh, R.K. (2006) A multi-objective genetic algorithm for scheduling in flow shops to minimize the makespan and total flow time of jobs. *International Journal of Advanced Manufacturing Technology, 27*, 804-815.

Rajendran, C. (1995) Heuristics for scheduling in flowshop with multiple objectives. *European Journal of Operational Research, 83*, 540-555.

Rajendran, C., & Ziegler, H. (2004) Ant-colony algorithms for permutation flowshop scheduling to minimize makespan/total flowtime of jobs. *European Journal of Operational Research, 155*, 426-438.

Ruiz, R., & Maroto, C. (2005) A comprehensive review and evaluation of permutation flowshop heuristics. *European Journal of Operational Research*, *165*, 479-494.

Sridhar, J., & Rajendran, C. (1996) Scheduling in flowshop and cellular manufacturing systems with multiple objectives—a genetic algorithmic approach. *Production Planning & Control, 7,* 374-382.

Taillard, E. (1993) Benchmarks for basic scheduling problems. *European Journal of Operational Research, 64*, 278-285.

Talbi, E.-G., Rahoual, M., Mabed, M.H., & Dhaenens, C. (2001) A hybrid evolutionary approach for multicriteria optimization problems: application to the flow shop. In Zitzler E. et al. (Eds.) *First International Conference on Evolutionary Multi-Criterion Optimization, EMO 2001. LNCS, 1993*, (pp. 416-428). Springer, Heidelberg.

Tseng, L.Y., & Lin, Y-T. (2009) A hybrid genetic local search algorithm for the permutation flowshop scheduling problem. *European Journal of Operational Research, 198*, 84-92.

Varadharajan, T.K., & Rajendran, C. (2005) A multi-objective simulated-annealing algorithm for scheduling in flowshops to minimize the makespan and total flowtime of jobs. *European Journal of Operational Research, 167*, 772-795.

Zhang, Y., Li, X., Wang, Q. (2009) Hybrid genetic algorithm for permutation flowshop scheduling problems with total flowtime minimization. *European Journal of Operational Research, 196*, 869–876.

Zhang, Q., & Li, H. (2007) MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation, 11*, 712-731.

Zitzler, E., Laumanns, M., & Thiele, L. (2001) SPEA2: improving the strength Pareto evolutionary algorithm. Technical Report 103, Computer Engineering and Network Laboratory (TIK), ETH, Zurich, Switzerland.

Zitzler, E., & Künzli, S. (2004) Indicator-based selection in multiobjective search. In X. Yao et al. (Eds.) *Parallel Problem Solving from Nature, PPSN VIII* (pp. 832-842).

Zobolas, G.I., Tarantilis, C.D., & Ioannou, G. (2009) Minimizing makespan in permutation flow shop scheduling problems using a hybrid metaheuristic algorithm. *Computers & Operations Research, 36*, 1249-1267.

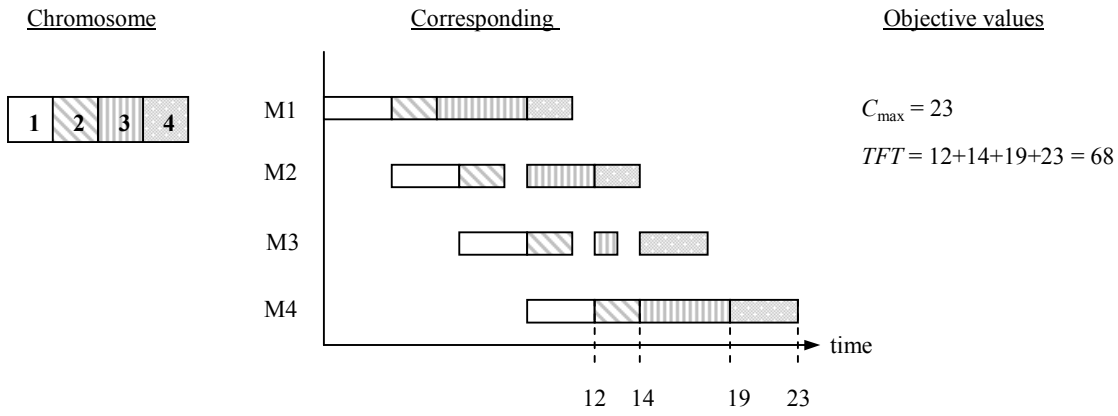


Fig. 1. An example of the encoding and decoding mechanism in the proposed NNMA

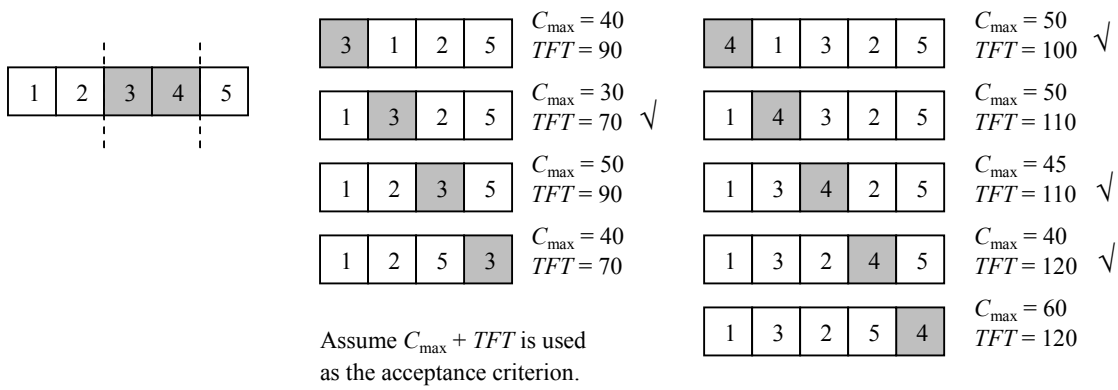


Fig. 2. An example of applying NEH-based local improving to an individual

Fig. 3. An example of segmentation in the NEH-based local improving (Assume that n_{NEH} is three.)

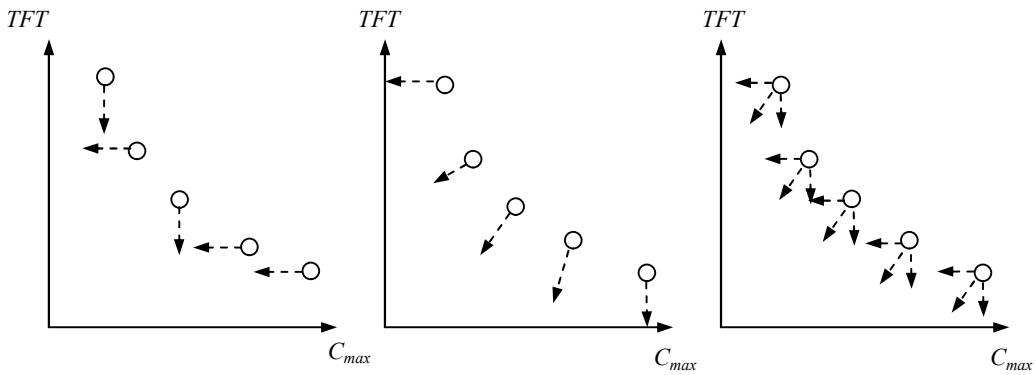


Fig. 4. Basic idea of the three proposed acceptance strategies in the NEH-based local improving procedure

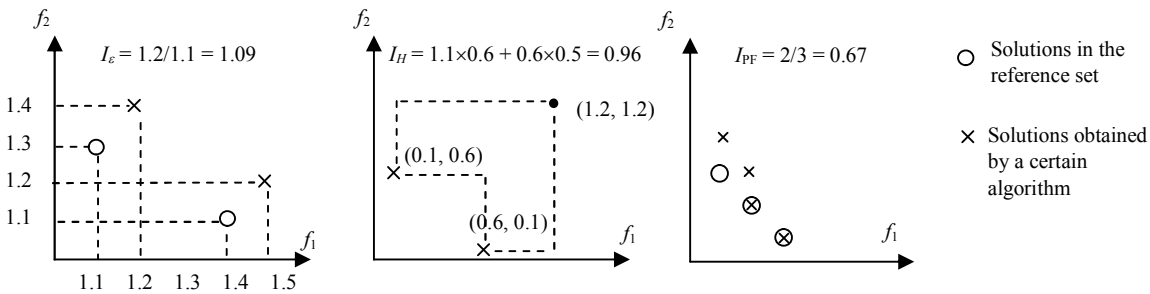
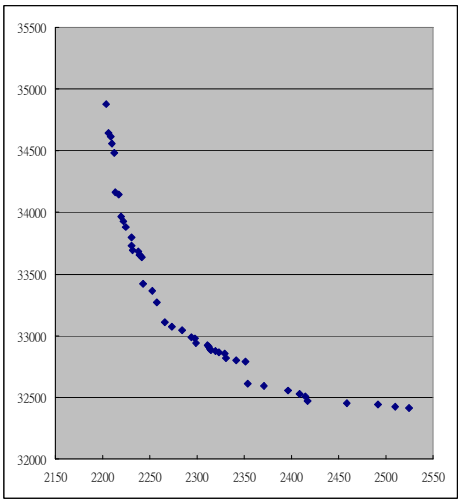
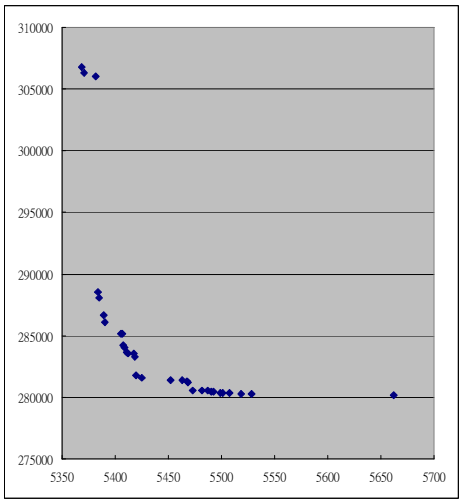


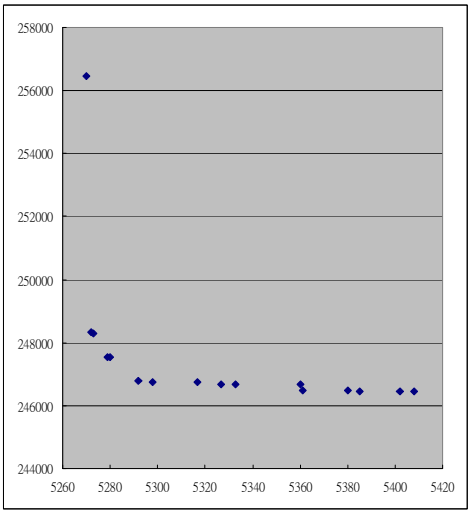
Fig. 5. Illustration of calculation of the three performance metrics in the experiment



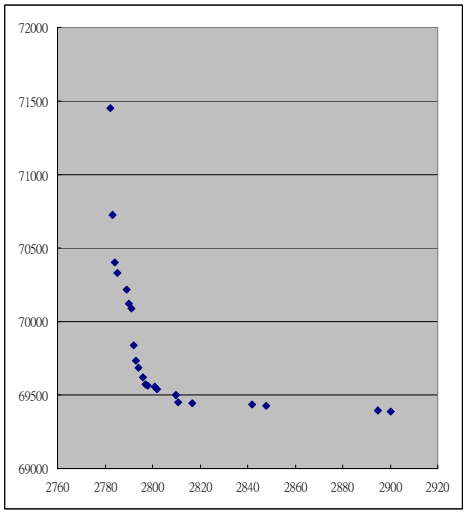
TA28 (smooth)



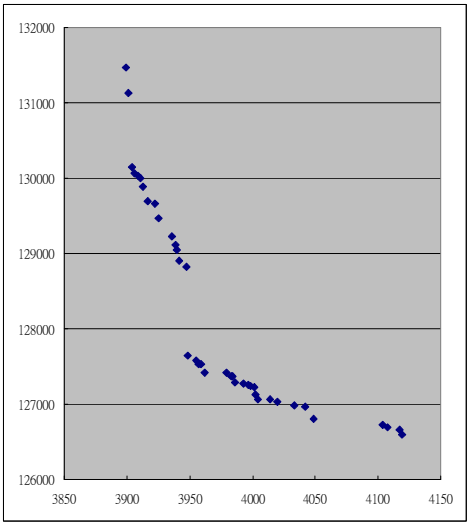
TA72 (large gap)



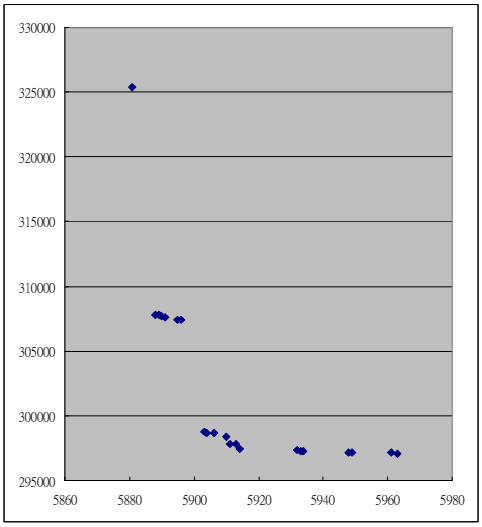
TA40 (*TFT*-easy)



TA62 (C_{max}-easy)



TA51 (clusters)



TA80 (many clusters)

Fig. 6. Typical distributions of the net sets of non-dominated solutions for TA instances

Algorithm 1

Algorithm 1. The **NNMA** Algorithm

t: current generation number P: population

p_i: parents o_i: offspring

r_m: mutation rate o_i': offspring after mutation

o_i^*: best individuals after environmental selection

T_L: interval to do local improving A: archive

S: set of individuals selecting from P for doing local improving

S^*: set of individuals obtained by doing local improving to individuals in S

T_G: the maximum number of generations

```
P ← Random_Initialize()
NSGAII_Evaluate(P)
t ← 0
repeat
   i ← 0
   repeat              // do reproduction
      p1 ← 2-Tournament_Select(P)
      p2 ← 2-Tournament_Select (P)
      {o1, o2} ← LOX_Crossover(p1, p2)
      o1' ← Swap_Mutate(o1, rm)
      o2' ← Swap_Mutate(o2, rm)
      NSGAII_Evaluate(P ∪ {o1', o2'})
      {o1*, o2*} ← EnvironmentalSelect(p1, p2, o1', o2', A)
      p1 ← o1*
      p2 ← o2*
      i ← i + 2
   until i = |P|
   if t mod TL = 0 then        // do local improving
      S ← LocalImprovingSelect(P, A)
      S* ← NEH_LocalImprove(S, t, TG)
      P ← Refill(P, S*)
   end if
   t ← t + 1
until t = TG or time is up
output A
```

Algorithm 2

Algorithm 2. The **NEH_LocalImprove** Procedure

N: set of selected neighboring complete solutions

t: current generation number T_G: the maximum number of generations

N_{NEH}: the maximum number of extracted jobs n_{NEH}: number of extracted jobs

S: set of individuals selecting from P for doing local improving (input to this procedure)

x: individual in S

$\sigma(x)$: permutation of jobs encoded in x

σ_U: permutation of jobs selected to be re-inserted

σ_S: permutation of jobs in $\sigma(x)$ but not in σ_U

π: re-ordered permutation of jobs in σ_U; jobs will be re-inserted one by one from the left to the right in this permutation

B: set of best partial solutions

D: set of neighboring partial solutions

$\pi[j]$: j^{th} job in the permutation π

σ_{jk}: permutation obtained by inserting $\pi[j]$ in the kth position in σ

S^*: set of individuals obtained by local improving (output from this procedure)

```
N ← ∅
n_NEH ← max{N_NEH·(1 − t/T_G), 1}
for each x in S
   {σ_U, σ_S} ← Segment(σ(x), n_NEH)
   π ← Reorder(σ_U)
   B ← {σ_S}
   for j = 1 to |π|                    // for each job to be re-inserted
      D ← ∅
      for each σ in B                  // for each best partial solution
         for k = 1 to |σ| + 1          // for each possible position to insert
            σ_jk = Insert(σ, π[j], k)
            D ← D ∪ {σ_jk}
         end for
      end for
      if j < |π| then
         B ← Accept(D)
      else
         B ← D
      end if
   end for          // j
   N ← N ∪ B
end for             // x
S* ← FindNondominatedSolutions(S ∪ N)
```

Table 1

An example of how the acceptance strategies work in the local improving procedure

Acceptance strategy	*D* in Algorithm 2	*B* in Algorithm 2
RO strategy (Assume C_{max} is chosen)		[1 3 2 5] (C_{max} = 30)
DW strategy (Assume weight vector is (0.5, 0.5))	[3 1 2 5] (40, 90) [1 3 2 5] (30, 70) [1 2 3 5] (50, 80) [1 2 5 3] (40, 60)	[1 3 2 5] (F = 0.5·30/40 + 0.5·70/90)
PD strategy		[1 3 2 5] [1 2 5 3]

Table 2

Setting and results in the experiment on deciding the timing of invocation, number of extracted jobs, and acceptance strategy for the local improving procedure

	TA11 – TA20 (20×10)	TA41 – TA50 (50×10)	TA71 – TA80 (100×10)
Population size	20	50	100
Generation number	40000	20000	6000
Mutation rate	0.05	0.05	0.05
Computation time (s)	16	40	80
Selection of individuals for local improving	Archive	Archive	Archive
Ordering of extracted jobs in local improving	Evolution	Evolution	Evolution
Timing of invocation (every T_L generations)	10, **20**, 40	**10**, 20, 40	**10**, 20, 40
Maximum number of extracted jobs (N_{NEH})	1, 2, **4**	2, **5**, 10	**5**, 10, 20
Change of number of extracted jobs (n_{NEH})	**Static**, Dynamic	Static, **Dynamic**	Static, **Dynamic**
Acceptance strategy	RO, **DW**, PD	RO, **DW**, PD	RO, **DW**, PD

Table 3

Results in the experiment on selection of individuals to do local improving

	TA11 – TA20 (20×10)		TA41 – TA50 (50×10)		TA71 – TA80 (100×10)	
	I_ε	I_{H}	I_ε	I_{H}	I_ε	I_{H}
Archive	1.044	1.161	**1.056**	**1.267**	**1.007**	**1.342**
2-tournament	**1.019**	**1.174**	1.064	1.240	1.200	1.092
Random	1.032	1.170	1.060	1.252	1.307	0.931

Table 4

Results in the experiment on ordering strategies in the local improving procedure

	TA11 – TA20 (20×10)		TA41 – TA50 (50×10)		TA71 – TA80 (100×10)	
	I_ε	I_{H}	I_ε	I_{H}	I_ε	I_{H}
Evolution	1.032	1.209	1.058	1.293	1.084	1.274
SPT	1.036	1.204	1.069	1.274	**1.078**	**1.282**
LPT	1.046	1.202	1.071	1.272	1.103	1.226
Random	1.039	1.206	1.056	1.299	1.089	1.262
Uniform	**1.020**	**1.213**	1.054	1.292	1.099	1.257
Cyclic	1.034	1.210	1.059	1.289	1.089	1.252
Stuck	1.030	1.211	**1.050**	**1.319**	1.084	1.269

Table 5

Results in the experiment on performance comparison with benchmark algorithms

		NNMA			Combo-23		
		I_ε	I_{H}	I_{PF}	I_ε	I_{H}	I_{PF}
TA01 – TA10	(20×5)	**1.016**	**1.089**	**92%**	1.024	1.083	85%
TA11 – TA20	(20×10)	1.018	1.160	**91%**	**1.015**	**1.162**	88%
TA21 – TA30	(20×20)	**1.029**	1.187	84%	1.030	**1.193**	**87%**
TA31 – TA40	(50×5)	**1.011**	**1.355**	**87%**	1.078	1.282	13%
TA41 – TA50	(50×10)	**1.002**	**1.343**	**98%**	1.120	1.177	2%
TA51 – TA60	(50×20)	**1.003**	**1.325**	**96%**	1.161	1.071	4%
TA61 – TA70	(100×5)	1.071	1.282	37%	**1.062**	**1.332**	**63%**
TA71 – TA80	(100×10)	**1.063**	**1.282**	**71%**	1.110	1.207	29%
TA81 – TA90	(100×20)	**1.006**	**1.351**	**99%**	1.200	1.051	1%

Table 6

Average deviation percentage from the best known solutions

	TA01-TA10 (20×5)	TA11-TA20 (20×10)	TA21-TA30 (20×20)	TA31-TA40 (50×5)	TA41-TA50 (50×10)
$C_{\max}$	0.05%	0.27%	0.20%	0.06%	0.92%
TFT	0%	0.00%	0.01%	0.59%	0.58%
	TA51-TA60 (50×20)	TA61-TA70 (100×5)	TA71-TA80 (100×10)	TA81-TA90 (100×20)	Average
$C_{\max}$	1.57%	0.07%	0.66%	2.22%	0.67%
TFT	0.70%	1.64%	1.29%	1.00%	0.65%

Table 7

Net set of non-dominated solutions for Taillard 50×10 problems (TA41– TA50)

TA41		TA42				TA43				TA44		TA45			
C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT
3035	91604	2913	86253	3125	83654	2878	86108	3070	80897	3071	89937	3010	93699	3144	86968
3036	90661	2919	85706			2879	85631	3072	80876	3074	89573	3011	93668	3177	86959
3038	90584	2926	85579			2880	85580	3113	80874	3076	89543	3013	93449	3184	86952
3039	90531	2929	85569			2881	85395	3115	80822	3079	89192	3015	93378	3185	86869
3046	89563	2930	85436			2882	85345	3124	80791	3081	89188	3018	89628	3218	86844
3047	89534	2933	85406			2883	85323			3082	89089	3020	89569	3230	86837
3065	89422	2934	85340			2884	84011			3083	88739	3023	89367	3237	86815
3066	89348	2936	85339			2887	83815			3084	88732	3024	89221		
3067	89152	2938	85228			2888	83535			3086	88708	3028	89057		
3070	88839	2947	85188			2896	83480			3087	88694	3029	88986		
3076	88466	2954	85066			2899	83392			3088	88539	3031	88965		
3092	88463	2960	84922			2904	83381			3089	87887	3032	88857		
3093	88426	2961	84869			2906	83335			3096	87741	3034	88842		
3137	88386	2962	84743			2916	83067			3098	87739	3035	88810		
3138	88317	2964	84723			2926	83027			3120	87729	3039	88776		
3149	88284	2973	84428			2927	83019			3123	87673	3042	88455		
3155	88257	2976	84406			2932	82967			3142	87635	3047	88409		
3157	88207	2990	84389			2941	82924			3145	87600	3049	88094		
3220	88201	2995	84361			2942	82867			3146	87569	3052	87909		
		3001	84314			2944	82627			3151	87533	3063	87888		
		3002	84295			2955	82350			3157	87404	3066	87870		
		3004	84269			2962	81781			3161	87395	3068	87857		
		3006	84257			2986	81772			3162	87343	3070	87837		
		3008	84239			2988	81711			3195	87297	3072	87799		
		3012	84233			2990	81528			3199	87246	3073	87637		
		3016	84200			3004	81315			3212	87224	3074	87497		
		3018	84154			3006	81182			3216	87213	3077	87464		
		3022	83918			3012	81069			3223	87212	3078	87372		
		3033	83904			3032	81012			3225	87166	3091	87284		
		3035	83898			3035	80997			3314	87163	3098	87264		
		3036	83886			3040	80975			3318	87150	3103	87213		
		3049	83743			3042	80963			3327	87149	3114	87098		
		3054	83739			3046	80955			3334	87111	3124	87071		
		3082	83699			3048	80948					3142	87057		
		3090	83698			3049	80916					3143	87014		

Table 7 (Continued)

Net set of non-dominated solutions for Taillard 50×10 problems (TA41– TA50)

TA46				TA47		TA48				TA49				TA50	
C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT
3019	91914	3215	87229	3124	92988	3039	93282	3208	87771	2910	91260	3037	86616	3108	92922
3030	91866	3217	87196	3125	92892	3043	92741	3253	87762	2912	90143	3043	86600	3109	92704
3034	91823	3238	87125	3126	91361	3055	91114	3256	87692	2913	90058	3045	86551	3111	92311
3038	91809	3240	87123	3131	91201	3060	90753	3266	87683	2914	89591	3059	86539	3118	92168
3039	91476	3291	87089	3144	91120	3062	90691	3319	87663	2916	89576	3060	86520	3128	91573
3042	91228	3301	87075	3149	90994	3063	90625	3326	87604	2917	89537	3061	86515	3132	91514
3043	90012			3150	90990	3064	90581			2924	89504	3063	86496	3136	90974
3046	89607			3154	90967	3065	90525			2928	89249	3065	86461	3137	90944
3047	89133			3156	90659	3068	90464			2929	89226	3127	86431	3138	90761
3049	89107			3158	90629	3080	90046			2932	89155	3129	86429	3142	90738
3052	89049			3170	90506	3081	89870			2936	89095	3136	86417	3143	90623
3053	89044			3182	90488	3083	89825			2939	89018	3154	86378	3148	90581
3055	89036			3183	90410	3087	89810			2941	88940	3156	86376	3150	90374
3056	89031			3208	90383	3088	89125			2942	88737	3163	86364	3154	90211
3060	88973			3220	90363	3090	89075			2943	88655	3188	86353	3158	90035
3061	88096			3230	90342	3094	89073			2945	88426	3191	86201	3159	89968
3069	88079			3233	90293	3098	88993			2949	88377	3192	86196	3161	89965
3072	88030			3238	90279	3105	88987			2951	88349	3250	86124	3163	89918
3073	87974			3239	90269	3108	88934			2956	88239			3169	89892
3075	87861			3241	90253	3109	88899			2957	88075			3171	89826
3077	87859			3251	90198	3114	88702			2961	88066			3175	89610
3079	87777			3270	90124	3116	88672			2962	87874			3180	89563
3089	87757			3276	90097	3119	88291			2964	87729			3204	89493
3094	87707			3282	90071	3120	88195			2968	87679			3209	89267
3095	87614			3287	90055	3121	88158			2977	87468			3212	89197
3096	87586			3288	90040	3145	88149			2985	87409			3223	89190
3099	87541			3329	89997	3146	88130			2993	87385			3225	89145
3102	87374			3352	89968	3159	88077			2999	87372			3228	89119
3140	87349			3354	89919	3161	88070			3002	87264			3236	89112
3152	87333			3372	89816	3162	88020			3005	87147			3239	89103
3175	87307					3172	88010			3017	87089			3241	88975
3181	87305					3174	87925			3023	87045			3282	88935
3186	87287					3192	87861			3025	86707			3449	88845
3209	87277					3198	87838			3029	86689				
3211	87260					3206	87828			3031	86640				

Table 8

Net set of non-dominated solutions for Taillard 50×20 problems (TA51– TA60)

TA51				TA52				TA53				TA54		TA55			
C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT
3899	131463	4104	126733	3762	126724	3933	120734	3693	121293	4036	117619	3775	125620	3679	123786	3832	119894
3901	131123	4108	126689	3764	123619	3936	120703	3694	121275			3777	125371	3680	122814	3835	119817
3904	130145	4117	126668	3774	123397	3954	120628	3697	121264			3778	124752	3683	122687	3850	119780
3906	130072	4119	126602	3781	122704	3997	120594	3701	120965			3785	124539	3687	122592	3852	119764
3909	130029			3783	122672	4020	120534	3702	120943			3793	124347	3694	122575	3859	119747
3911	129992			3787	122517			3703	120936			3795	123532	3697	122567	3873	119708
3913	129882			3789	122472			3705	120627			3809	123465	3698	122507	3876	119706
3917	129686			3792	122395			3712	120499			3814	123262	3699	122402	3877	119630
3922	129659			3797	122264			3713	119843			3815	123205	3703	122323	3887	119629
3925	129460			3799	122110			3714	119841			3817	123190	3706	122246	3947	119626
3936	129221			3801	121739			3715	119830			3818	123065	3707	122213	3954	119620
3939	129114			3812	121656			3722	119436			3820	122825	3708	122150	3960	119612
3940	129052			3816	121634			3723	119389			3826	122538	3709	121984	3983	119595
3942	128904			3818	121589			3724	119298			3829	122439	3713	121817		
3947	128822			3820	121563			3728	119275			3831	122434	3716	121602		
3948	127644			3821	121430			3732	119241			3833	122225	3720	121485		
3955	127574			3825	121412			3734	119240			3843	122144	3728	121319		
3957	127535			3839	121245			3740	119025			3853	122056	3729	121158		
3959	127527			3852	121242			3748	119023			3854	121982	3744	121122		
3962	127421			3859	121177			3751	118890			3872	121969	3749	121050		
3979	127415			3863	121129			3756	118789			3880	121905	3750	121010		
3983	127375			3874	121122			3759	118575			3881	121884	3757	120986		
3984	127371			3876	121113			3761	118238			3893	121876	3759	120930		
3986	127296			3879	121066			3790	118187			3896	121872	3766	120874		
3993	127267			3880	121043			3794	117988			3909	121796	3768	120806		
3997	127256			3883	121037			3810	117979			3912	121785	3770	120725		
3999	127250			3886	121029			3814	117953			3913	121760	3771	120475		
4001	127222			3887	121024			3815	117904			3914	121709	3772	120253		
4002	127137			3888	121000			3845	117902			3921	121681	3785	120203		
4004	127067			3891	120945			3846	117854			3923	121589	3814	120160		
4014	127059			3895	120875			3925	117830			3939	121553	3816	120146		
4020	127039			3911	120859			3996	117771					3818	120109		
4033	126983			3912	120849			3999	117769					3820	120077		
4042	126969			3913	120778			4000	117732					3821	120011		
4049	126808			3914	120752			4032	117690					3824	120009		

Table 8 (Continued)

Net set of non-dominated solutions for Taillard 50×20 problems (TA51– TA60)

TA56				TA57				TA58				TA59		TA60			
C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT
3739	129239	3872	122450	3769	126508	4037	124166	3763	127351	3925	124012	3816	127111	3786	127113	4103	125203
3740	129174	3875	122423	3770	126500	4038	124146	3766	126350	3946	123913	3817	127046	3788	127078	4115	125196
3741	128940	3881	122077	3775	126479	4066	124115	3774	126168	3952	123888	3823	126798	3789	126968	4129	125166
3744	128895	3884	122022	3776	126476			3783	126101	3956	123880	3824	126533	3794	126894	4144	125146
3746	128861	3893	122009	3778	126254			3785	126024	3964	123809	3828	126306	3795	126870	4150	125123
3747	127687	3894	122002	3784	126187			3786	125874	3976	123791	3831	126283	3797	126856		
3751	127242	3908	121995	3787	126037			3789	125778	3980	123762	3834	126085	3802	126739		
3752	127004	3917	121986	3790	125689			3800	125769	3983	123759	3849	126016	3805	126724		
3756	126073	3932	121869	3794	125671			3801	125711	4041	123748	3850	126015	3809	126683		
3759	125055	3940	121808	3797	125389			3802	125643	4053	123722	3853	125814	3811	126649		
3762	124716	3951	121763	3803	125369			3807	125445			3854	125805	3817	126630		
3768	124570	3955	121698	3804	125332			3808	125404			3863	125677	3820	126623		
3769	124561	4089	121688	3805	125248			3810	125385			3864	125597	3821	126563		
3770	124384			3818	125234			3815	125348			3868	125543	3823	126519		
3773	124248			3821	125172			3821	125232			3869	124923	3828	126458		
3778	124166			3834	125138			3822	125227			3878	124917	3831	126212		
3779	124132			3836	125081			3832	125222			3879	124854	3839	126187		
3784	124105			3840	125055			3833	124995			3880	124763	3847	126178		
3789	124094			3846	125046			3843	124985			3887	124501	3855	126175		
3800	123455			3847	124887			3851	124952			3891	124435	3864	126022		
3808	123416			3848	124838			3855	124810			3892	124215	3879	125910		
3813	123388			3849	124811			3858	124585			3893	124082	3881	125827		
3816	123367			3850	124710			3876	124576			3897	123861	3883	125780		
3819	123320			3856	124589			3877	124524			3899	123779	3886	125745		
3820	123199			3888	124476			3880	124470			3901	123685	3888	125608		
3827	123082			3900	124429			3882	124457			3903	123662	3890	125566		
3831	123014			3904	124413			3883	124438			3904	123609	3900	125415		
3832	122940			3912	124362			3885	124391			3907	123548	3917	125369		
3840	122730			3913	124329			3890	124338			3909	123539	3919	125344		
3841	122644			3916	124326			3898	124272			3911	123509	3972	125319		
3852	122557			3918	124241			3901	124192			3916	123475	3990	125305		
3858	122495			3950	124224			3903	124153			3929	123424	3998	125289		
3866	122489			3955	124194			3904	124128			3932	123399	4022	125269		
3868	122484			3968	124170			3913	124107			3933	123361	4029	125234		
3870	122469			3971	124169			3923	124087			3945	123203	4062	125222		

Table 9

Net set of non-dominated solutions for Taillard 100×20 problems (TA81– TA90)

TA81				TA82				TA83				TA84				TA85			
C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT
6380	392227	6494	375550	6324	387911	6452	381195	6400	398350	6534	379619	6367	395334	6504	381383	6417	387160	6594	377821
6382	389193	6497	375316	6327	387137	6454	381129	6401	398340	6535	379575	6369	393609	6507	381374	6418	384687	6604	377793
6385	386802	6498	375283	6332	386531	6455	381096	6402	398241	6538	379573	6373	393447	6508	381337	6419	383709	6608	377789
6386	386781	6501	375171	6338	386127	6456	381091	6405	397409	6540	379560	6376	393409	6531	381325	6420	383673	6618	377782
6389	386628	6502	375017	6339	385192	6457	380580	6407	397369	6542	379558	6377	393207	6533	381244	6421	383620	6621	377610
6390	384231	6524	374954	6341	384957	6458	380540	6411	391805	6543	379545	6381	392519	6545	381239	6424	383607	6628	377480
6399	384220	6525	374876	6342	384756	6461	380508	6412	391309	6545	379543	6382	391268	6550	381204	6425	383601	6634	377369
6404	384069	6526	374788	6346	384706	6475	380473	6419	391084	6552	379449	6389	391153	6554	381200	6433	383587	6651	377305
6408	382666	6528	374732	6348	384613	6478	380398	6425	390733	6570	379419	6393	391114	6557	381195	6445	383575	6653	377083
6413	382299	6529	374649	6349	384188	6480	380291	6426	390394	6571	379377	6398	391109	6558	380902	6446	382960	6660	376981
6417	381871	6532	374556	6351	384120	6481	380273	6428	387434	6573	379356	6401	389468	6559	380863	6447	382901	6664	376934
6418	381575	6535	374502	6353	383996	6483	380262	6430	387433	6576	379316	6402	389360	6564	380848	6452	382207	6671	376879
6419	381478	6536	374490	6357	383440	6485	380252	6431	386796	6577	379277	6403	389029	6568	380733	6455	382072	6683	376866
6421	381347	6538	374450	6359	383104	6507	379590	6442	384926	6578	379272	6413	388963	6570	380731	6457	381618	6696	376293
6422	381332	6543	374390	6360	383089	6526	379508	6455	384259	6593	379146	6423	388053	6575	380721	6468	381304	6704	376015
6423	381266	6573	374367	6361	383017	6580	379495	6456	383911	6610	379003	6427	387025	6577	380719	6478	381198		
6425	381003	6575	374336	6364	382936	6598	379481	6457	382498	6611	378916	6430	386736	6584	380493	6479	381037		
6428	380219	6576	374321	6370	382784	6610	379466	6458	382354	6617	378901	6432	386625	6663	380445	6489	380804		
6432	379664	6578	374270	6376	382778	6612	379411	6461	382312	6621	378851	6433	386480	6689	380412	6498	380433		
6433	378298	6592	374246	6378	382701	6615	379391	6463	382246	6633	378827	6434	386394	6696	380403	6500	380251		
6436	378246	6611	374233	6379	382691	6617	379377	6464	382221	6634	378751	6437	386382	6712	380377	6504	380141		
6437	378245	6617	374089	6380	382607	6620	379371	6465	382143	6636	378677	6441	386367	6728	380272	6509	380131		
6438	378236	6622	374064	6388	382559	6621	379275	6466	382096	6637	378610	6443	386210	6737	380233	6512	380112		
6441	378137	6628	373911	6389	382363	6654	379267	6471	382075	6661	378596	6444	386135	6749	380220	6513	379505		
6449	378098	6791	373896	6391	382354	6657	379219	6472	381841	6666	378546	6451	385991	6753	380197	6516	379098		
6450	377940	6813	373851	6400	382344	6667	379204	6477	381819	6667	378413	6452	385951	6817	380166	6530	379039		
6452	377807			6402	382335	6714	379192	6482	381033	6669	378367	6454	384239			6543	378649		
6459	377747			6411	382314			6484	380956	6672	378338	6468	384104			6549	378635		
6464	377706			6413	382305			6485	380952	6675	378263	6471	383780			6550	378574		
6465	377318			6421	382064			6490	380925	6706	378221	6477	383461			6556	378560		
6467	377112			6428	381976			6510	380562	6729	378191	6495	382160			6558	378529		
6468	376889			6431	381790			6511	380454	6750	378149	6497	381874			6566	378184		
6484	376879			6435	381403			6515	379913	6767	378116	6498	381804			6568	378048		
6486	376467			6439	381360			6525	379856			6499	381795			6581	378007		
6492	376163			6449	381349			6533	379813			6503	381393			6584	377898		

Table 9 (Continued)

Net set of non-dominated solutions for Taillard 100×20 problems (TA81– TA90)

TA86				TA87				TA88		TA89		TA90	
C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT	C_{max}	TFT
6518	390673	6634	379731	6434	395459	6610	382028	6560	400185	6415	388824	6544	405288
6522	390622	6636	379602	6436	392995	6614	381966	6561	400040	6420	388798	6564	402220
6526	390598	6641	379464	6439	392547	6632	381925	6562	399014	6421	388501	6565	396627
6527	390595	6643	379460	6442	392435	6634	381853	6567	398997	6422	388491	6568	394601
6528	389593	6656	379359	6447	392426	6672	381743	6577	398742	6428	388450	6570	394277
6530	387314	6658	379349	6449	392179	6684	381694	6588	398630	6433	387966	6581	393826
6531	387301	6663	378943	6450	392121	6726	381627	6598	397914	6434	387868	6582	393548
6535	385621	6670	378563	6453	389410	6736	381586	6613	396286	6441	387557	6587	393412
6539	385106	6673	378192	6457	388294	6745	381549	6615	396065	6442	387538	6589	391287
6542	384297	6689	378149	6458	388171	6780	381544	6618	396062	6448	387091	6603	390700
6543	384226	6696	378143	6459	388162	6786	381517	6625	396042	6449	387079	6612	390141
6544	384156	6697	378039	6460	388102			6627	395851	6451	384482	6619	389907
6551	384106	6773	377972	6471	388086			6632	395826	6454	384399	6626	389226
6552	383736	6792	377967	6474	388058			6635	395710	6462	384386	6629	389142
6553	383693			6484	387970			6644	395658	6469	384284	6633	389125
6554	381638			6485	387428			6646	395571	6473	384272	6639	389089
6564	381493			6491	387272			6647	395424	6476	383448	6640	388923
6565	381415			6492	386820			6649	395406	6477	383375	6652	388905
6567	381232			6495	386485			6652	395374	6495	383292	6663	388799
6568	381154			6503	384242			6656	395345	6507	383102	6664	388797
6569	381074			6512	383862			6661	394529	6509	382977	6666	388022
6570	380975			6517	383566			6683	394494	6512	382906	6668	387993
6571	380879			6524	383256			6702	394475	6522	382512	6671	387916
6576	380828			6526	383240			6750	393779	6532	382510	6683	387637
6587	380583			6528	383238			6755	393741	6534	382344	6689	387456
6594	380525			6529	382916			6769	393537	6538	381904	6696	387455
6599	380455			6533	382766			6784	393434	6562	381885	6706	386431
6604	380452			6551	382727			6800	393268	6566	381858	6761	386307
6605	380305			6552	382686			6827	392957	6567	381749	6783	386304
6609	380031			6559	382628			6852	392940	6607	381734	6786	386286
6617	380028			6564	382610			6859	392846	6725	381700	6879	386194
6620	379890			6582	382522			6865	392825	6768	381590	6927	386032
6625	379835			6587	382348			6872	392787	6805	381511	7009	386017
6630	379817			6595	382333			6912	392513	6902	381488	7010	385992
6632	379772			6605	382112								